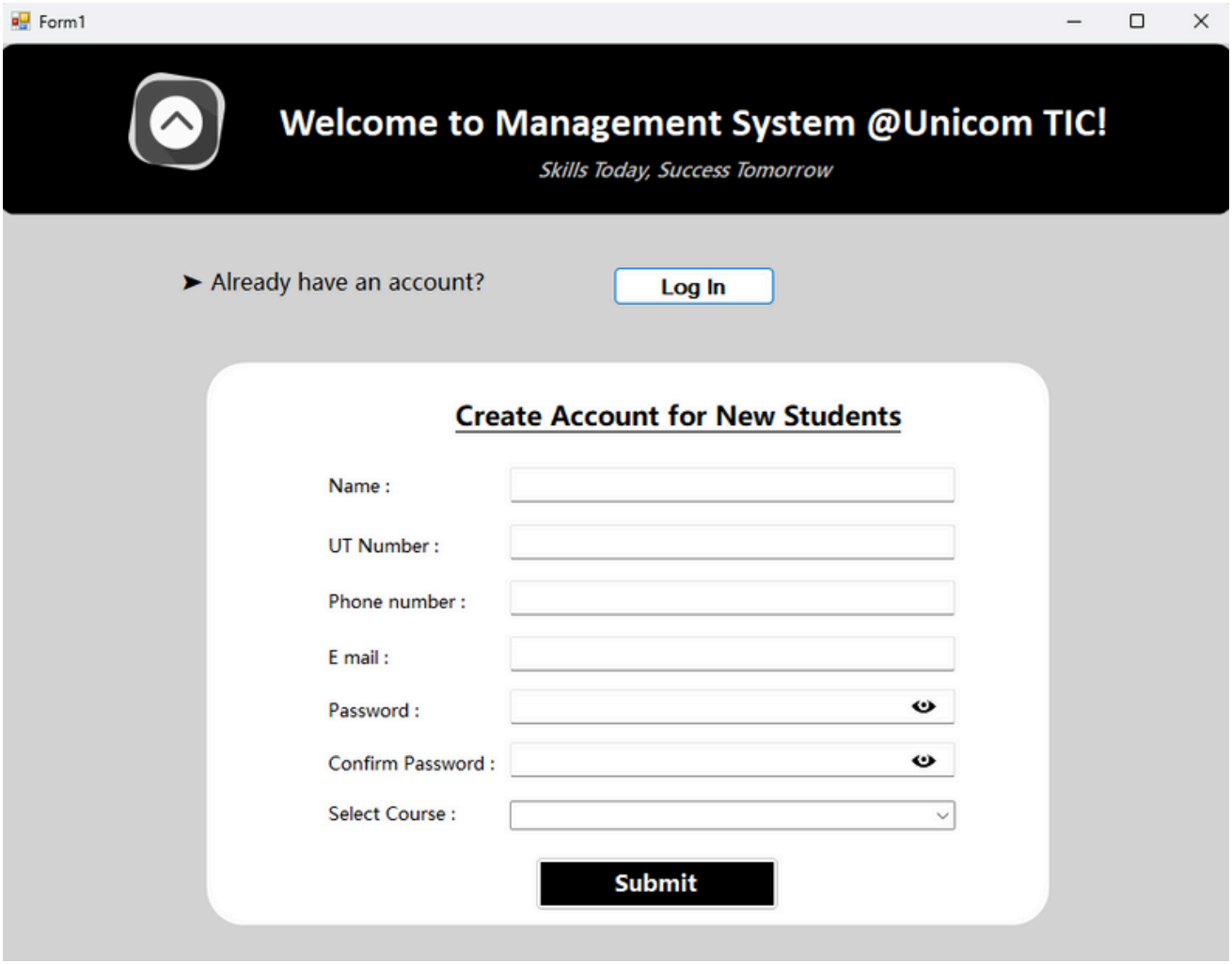
Unicom TIC
Training & Innovation Centre
Skills Today, Success Tomorrow

UnicomTIC Management System

PROJECT REPORT

Name : Pravika Ravi
UT Number : UT010008

FRONT PAGE


Form1
Welcome to Management System @Unicom TIC!
Skills Today, Success Tomorrow
Already have an account?
Log In
Create Account for New Students
Name :
UT Number :
Phone number :
E mail :
Password :
Confirm Password :
Select Course :
Submit

Contents

1.Project Overview

The UnicomTIC Management System (UMS) is a C# WinForms-based desktop application designed to manage key operations in an educational institution. It supports role-based access for Admin, Staff, Lecturers, and Students. Each role has access to features like student management, exams, timetable generation with room assignments, feedback, leave requests, and more.

2. Technologies Used

- Programming Language: C#
- Framework: WinForms (Windows Forms)
- Architecture Pattern: MVC (Model–View–Controller)
- Database: SQLite using System.Data.SQLite
- IDE: Visual Studio
- Design Focus: Clean MVC logic and role-based modules

3. Project Development History

Initially, the project began without fully applying the ER diagram due to carelessness. Backend logic was implemented without establishing proper table relationships. Upon realizing the issue halfway through development, I attempted to fix it—but due to confusion, I decided to restart the project from scratch.

This time, I carefully designed the ER diagram, implemented proper table relationships with foreign keys, and restructured the project following the MVC pattern strictly. Instead of learning unfamiliar technologies, I focused on refining features and building out complete modules.

4. System Modules and Role-Based Features

Client Requirements

Admin Features:

- Add/Edit/Delete:
  - Courses, Subjects, Students, Exams, Timetables, Users, Rooms
- Mark Management
- Attendance Entry
- Manage Signup requests

Lecturer Features:

- View Timetable
- View Student Marks
- Change Own Password

Staff Features:

- View Timetable
- Add/Edit/Delete Exams
- Add/Edit Marks
- Change Own Password

Student Features:

- View Own Profile
- View Timetable
- View Exam and Marks
- Request Leave
- Change Own Password

Additional Features

1.Signup Request System

Overview
Students submit their signup requests via a form, and the data is stored temporarily. Admin can either Accept or Decline the request.

Workflow
Student Submission: The form data is stored in a Temporary Table.
Admin Actions:

- Accept: Data is moved to the Permanent Table.
- Decline: Data is deleted permanently from the temporary table.

2. Leave Request System

Overview
Students submit leave requests via a form, and the data is stored temporarily. Admin can either Approve or Reject the request.

Workflow
Student Submission: The leave request data is stored in a Temporary Table.
Admin Actions:

- Approve: Data is moved to the Permanent Table.
- Reject: Data is deleted permanently from the temporary table.

3. Study Material Feature

Overview

Lecturers can upload study materials (such as PDFs, videos, or documents) for students to access. The goal was to allow students to view these materials within a specified section.

Workflow

Lecturer's Upload: Lecturers can add study materials via a form (e.g., uploading files and adding descriptions).
Student's View: Students can browse and view available study materials.

Pending Tasks

Lecturer Upload Interface: Completed (but not fully integrated).
Student Viewing: Not yet completed due to time constraints.

5. Database Schema Summary

- **Users**: Stores login credentials and roles.
- **Courses & Subjects:** Courses contain multiple subjects.
- **Students:** Linked to courses via CourseID.
- **Exams & Marks:** Each subject can have multiple exams; marks are stored in the ExamResults table.
- **Rooms:** Stores data on lecture halls and labs.
- **Timetables:** Schedules classes by subject, room, lecturer, and time.
- **LeaveRequests:** Students can submit leave requests, stored in the LeaveRequests table.
- **StudyMaterials:** Linked to subjects and lecturers; stores study material uploads.

All tables are properly connected using foreign key relationships. Data is stored in SQLite.

6. Challenges Faced

- Missed foreign keys in initial design — led to a full project restart.
- Unfamiliar with database management due to missing some DB classes.
- Solved by restarting via ER diagram and connecting tables properly.
- Large feature scope and limited time made it hard to finish all ideas
.
- Learning curve for WinForms + MVC + async operations slowed early development.

7. Learning Outcomes

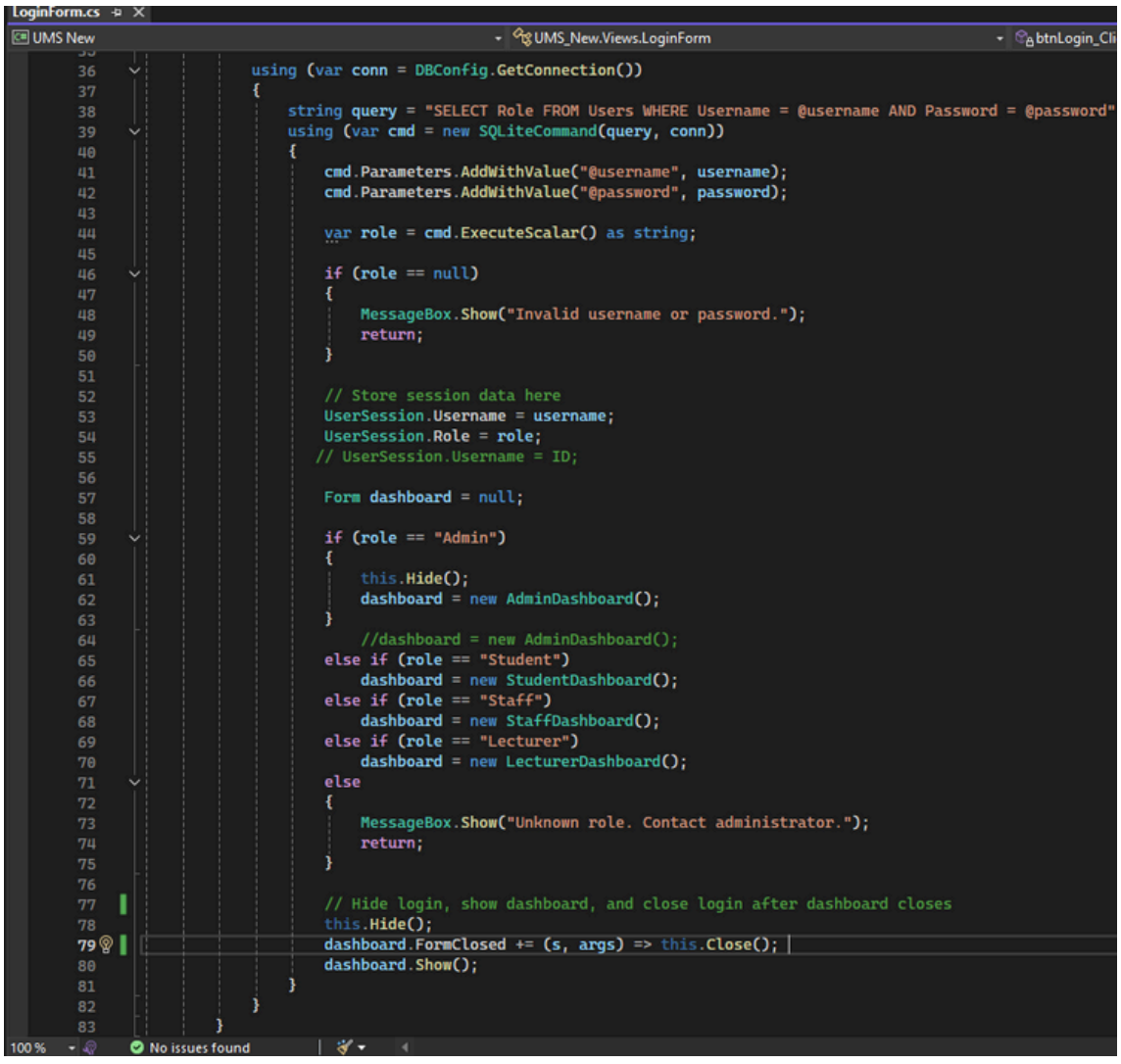
- Gained deep hands-on experience with C# WinForms and MVC.
- Learned to design an ER diagram and properly implement it in a real system.
- Understood the value of proper database design before coding.
- Learned modular UI design using UserControl + dynamic form loading.
- Built real-world features with role-based control, data validation, and clean structure.

8. Future Improvements

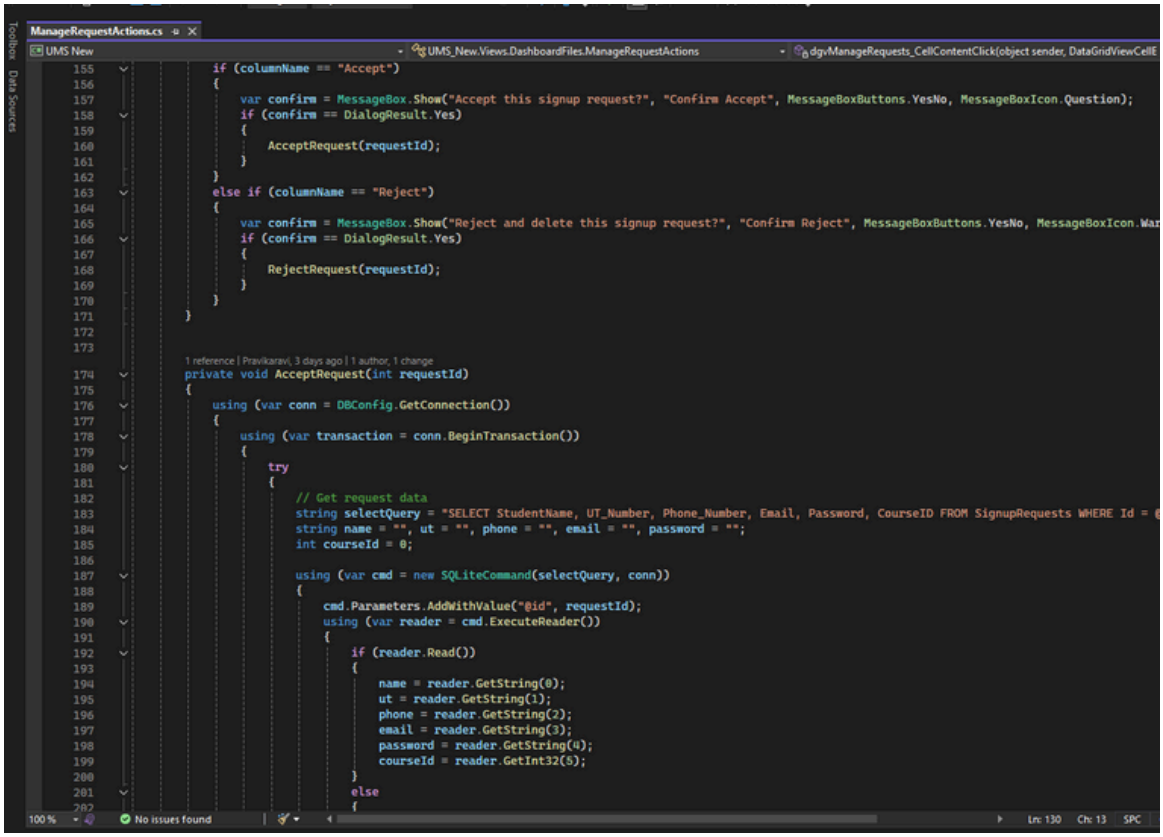
- Timetable conflict detection (room/time clashes)
- Password hashing instead of plaintext
- Advanced report generation and charts
- Improve UI visuals with modern design patterns
- Add backup/export feature for data

9. Screenshots & Code Samples

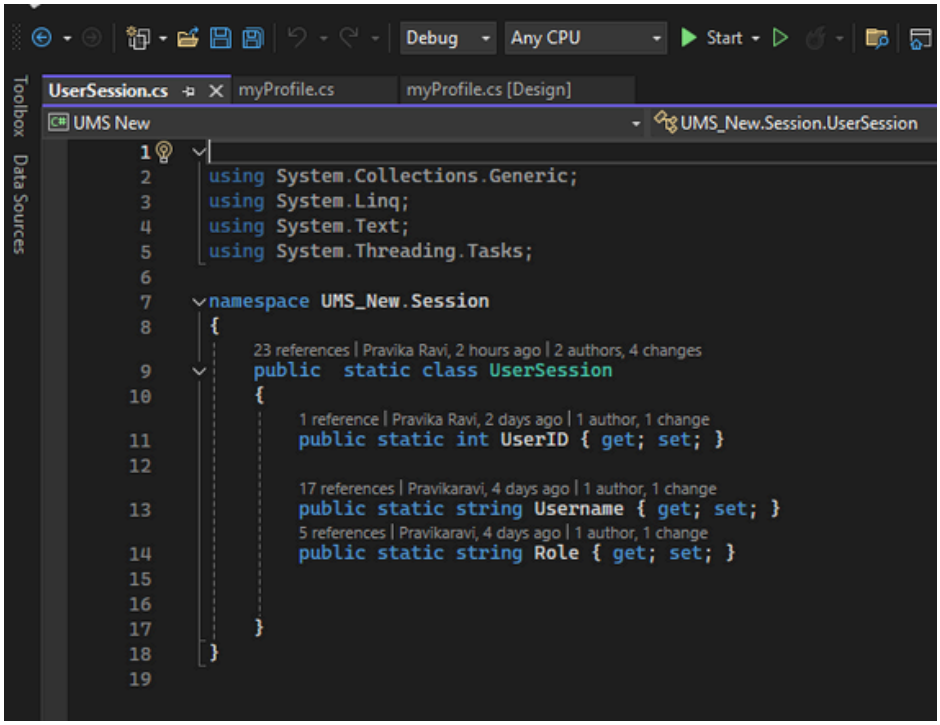
LoginController (role-based redirection)



Signup request handling



Using static class Session



10. Final Notes

This project helped me grow in both confidence and skill with real software development principles. While I couldn't implement every idea due to time and personal challenges, I'm proud of the foundation I built and the features I completed.